

Autonomous Multi-Agent Collaboration

GROUP01

Diego Correa Oliver Bielke

20011101 20031007

dcorrea@kth.se obielke@kth.se



Abstract

This report presents our solution for an autonomous multi-agent Pac-man capture-the-flag game, where two teams compete to collect more enemy pills within a fixed time limit. Although the task is implemented as a game, it captures important challenges from real multi-agent robotic systems, such as agent coordination, navigation around obstacles and other agents, planning around danger zones, and deciding when to pursue a goal or retreat. These problems are relevant in applications such as warehouse automation and autonomous delivery, where multiple agents must act safely and efficiently in shared environments. Our approach combines role-based behavior trees, A* path planning, particle-filter-based enemy tracking, and Voronoi partitioning for safety evaluation. Agents are assigned attacker or defender roles, where attackers search for safe enemy pills and return them to the base, while defenders guard important lanes and react to intruders. Across the two competition rounds, our team achieved the highest combined average score across the evaluated maps. These results suggest that combining high-level behavior selection with safety-aware path planning is an effective strategy for multi-agent collaboration under uncertainty.

1 Introduction

Autonomous multi-agent systems are becoming increasingly important in applications where several robots must operate in the same environment. Examples include warehouse robots moving goods, autonomous delivery vehicles sharing roads and sidewalks, drones inspecting large areas, and rescue robots searching dangerous environments. In all of these settings, each agent must make decisions based not only on its own objective, but also on obstacles, teammates, opponents or dynamic agents, and incomplete information about the environment. A good multi-agent navigation strategy must therefore combine goal-seeking behavior with safety, coordination, and adaptation.

The Pacman capture-the-flag environment used in this assignment provides a simplified but useful version of this problem. Two teams compete for a fixed time, and the objective is to collect more pills from the opponent's side than the opposing team collects from ours. The difficulty comes from the fact that an agent changes role depending on its position on the map. When entering enemy territory, an agent acts as Pacman and becomes vulnerable to enemy ghosts. When staying on its own side, it acts as a ghost and can defend against enemy Pacman agents. This creates a natural trade-off between attacking, defending, retreating, and reacting to danger.

The problem is challenging because the agents must operate under partial observability and time pressure. Enemy agents are not always visible, and decisions must be made from uncertain estimates of their positions. Attackers should collect pills efficiently, but they must also avoid being captured and should return safely when carrying enough pills. Defenders should protect lanes of the map, but they should also intercept intruders. A purely greedy strategy, such as always moving toward the closest pill, can easily fail if it ignores enemy positions. Similarly, a purely defensive strategy may survive but fail to score enough points.

In this report, we present a role-based multi-agent solution that combines high-level decision making with safety-aware path planning. Our agents are assigned attacker or defender roles. Attackers select pills and retreat paths using safety information derived from enemy positions, while defenders guard important lanes and react to visible or estimated intruders. Behavior trees are used to organize the decision logic in a modular way, allowing each agent to switch between behaviors such as attacking, retreating, defending, chasing, and returning to a safer position. For navigation, the agents use A* search on an obstacle map. To handle uncertainty about hidden enemies, particle-filter-based tracking is used to estimate likely enemy positions. These estimates are then used together with Voronoi partitioning to reason about safer and more dangerous regions of the map.

The final system performed strongly in the competition setting. Across the two competition rounds, our team achieved the highest combined average score across the evaluated maps.

1.1 Outline

The rest of the report explains how this result was achieved and how it compares to related approaches. Section 2 discusses previous work on behavior trees, particle filters, and safe path planning, which provide the main ideas used in our solution. Section 3 describes the proposed method, including the role assignment, behavior trees, path planning, enemy tracking, and safety evaluation. Section 4 presents the competition results and analyzes why the method performed well on the evaluated maps. Finally, Section 5 summarizes the main findings and discusses possible improvements.

2 Related work

This section reviews previous work related to the main components of our solution. The proposed system combines high-level behavior selection, uncertainty-aware enemy tracking, and safety-aware path planning. Behavior trees are relevant because they provide a modular way to organize reactive agent behavior. Particle filters are relevant because they allow agents to reason about uncertain enemy positions under partial observability. Finally, Voronoi partitioning is relevant because it can be used to reason about spatial safety and control regions in multi-agent environments.

2.1 Behavior trees

Behavior trees have been widely used to structure the decision-making logic of autonomous agents. Colledanchise and Ögren describe behavior trees as a method for organizing how an artificial agent switches between tasks, with applications in both robotics and game AI. A key advantage of behavior trees is that they are modular and reactive, which makes them suitable for systems where an agent must repeatedly choose between different behaviors depending on the current state of the environment [3].

This is closely related to our work because our agents needed to switch between several modes, such as attacking, retreating, and defending. A flat set of if-statements or a large finite-state machine would make this logic difficult to extend and debug. In contrast, behavior trees allowed us to

separate the logic into smaller behavior modules and combine them into attacker and defender trees.

Iovino et al. survey the use of behavior trees in robotics and artificial intelligence, noting that behavior trees were originally popularized in computer games and later adopted in robotics because they scale better than many finite-state-machine implementations when agent behavior becomes complex [6]. This motivation also appeared in our implementation. The Pacman capture-the-flag task required agents to react to dynamic events, such as seeing an enemy, becoming scared, carrying pills, or detecting that the match was close to ending. Behavior trees provided a convenient structure for encoding these priorities while keeping the implementation readable.

2.2 Particle filters

Particle filters are commonly used for state estimation when the state of a system cannot be directly observed. Instead of representing uncertainty with a single estimate, a particle filter represents a probability distribution using a set of weighted samples. Dellaert et al. introduced Monte Carlo Localization for mobile robots, where samples are used to approximate the robot's belief over possible poses and recursively update that belief using motion and sensor information [4]. This idea is important for robotics because real agents often need to act without complete knowledge of the world.

Thrun also describes particle filters as a central method in robotics, especially for localization problems where the belief distribution may be complex or multi-modal [7]. This is relevant for our project because hidden enemy agents can occupy several possible positions when they are outside the visible range. A single deterministic estimate would lose this uncertainty, while a particle filter can maintain several hypotheses about where an enemy may be.

In our solution, particle filters were used to estimate enemy positions under partial observability. When an enemy was visible, the estimate could be corrected using the observation. When the enemy was hidden, the particles were propagated according to possible movement, allowing the team to maintain a belief over likely enemy locations. These estimates were then used by other parts of the system, such as defender reactions and safety-aware path planning.

2.3 Voronoi partitioning

Voronoi partitioning is a fundamental spatial decomposition technique that divides a plane into regions based on proximity to specific sites [1]. In

robotics, these diagrams are widely used to establish navigation roadmaps that maximize clearance from obstacles, ensuring agents remain at a safe distance from potential collisions [2].

In multi-agent systems, graph-based Voronoi regions can be utilized to optimize agent positioning with the goal of maximizing survival probabilities rather than simple distance metrics [8]. This conceptualization of spatial “ownership” allows for a more nuanced evaluation of safety in adversarial environments. Recent research has shown that this strategy can be extended by treating Voronoi-derived safety data as a dynamic layer within a hierarchical control architecture [5]. Our implementation adopts this paradigm by using a two-team Voronoi partition to generate a normalized “danger score,” which is then integrated into the A* pathfinding stack as a cost penalty to prioritize survival during navigation.

3 Method

In this section, we describe the theoretical approach and the technical implementation of our multi-agent navigation system. Our solution is designed to handle the dynamic challenges of the Pacman capture-the-flag environment, focusing on autonomous role switching, state estimation under partial observability, and safety-aware path planning.

3.1 Theoretical Framework

The conceptual foundation of our agent relies on three primary methods: modular control for decision making, Bayesian filtering for tracking, and spatial partitioning for safety.

3.1.1 Reactive Behavior Control

To manage the complexity of an agent that must transition between offensive and defensive roles, we utilize a modular control architecture. A key challenge in this domain is “task switching”—deciding when to pursue a goal and when to retreat to safety. Behavior Trees (BTs) were selected as the primary mechanism because they offer a modular and reactive way to organize these tasks. Unlike traditional finite-state machines, BTs are composed of hierarchical nodes that allow for clear prioritization, ensuring that safety-critical conditions are always evaluated before goal-seeking tasks.

3.1.2 Probabilistic State Estimation

The game environment is characterized by partial observability, where exact enemy positions are only revealed through line-of-sight, while obscured agents provide only noisy signals based on their movement speed. To handle this uncertainty, we apply the principles of Bayesian filtering. We model the enemy’s state as a probability distribution over the map, which allows the agent to maintain multiple hypotheses about the opponent’s location even when they are hidden in the “fog of war.” We utilize a particle filter approach, representing the state distribution using a set of discrete samples (particles) that are propagated through a motion model and updated based on sensor observations.

3.1.3 Spatial Safety Evaluation

To bridge the gap between high-level roles and low-level path planning, we use Voronoi partitioning for spatial reasoning. By partitioning the traversable space into regions based on the distance to the nearest agent, we can identify “danger zones” where an enemy has a high probability of intercepting our agent. This allows the navigation algorithm to weigh the shortest path against the safest path.

3.2 Implementation Details

This subsection details how the theoretical methods described above were specifically configured and integrated into our final agent.

3.2.1 System Architecture and the Blackboard

Our implementation utilizes a “Blackboard” pattern to centralize data. The Blackboard stores the current match state, including the agent’s role, pill carry count, and the coordinates of all enemies. Every update cycle, the main AI script refreshes these values, which are then queried by the behavior trees and the safety module to ensure all components act on the same information.

3.2.2 Behavior Tree Configuration

Agents are assigned either an *Attacker* or a *Defender* tree. The Attacker Tree is structured with a selector node at the root to check priorities in the following order:

- **Late Game Return:** Triggered if match time is below a threshold to secure points.

- **Power Capsules:** Prioritizes acquiring capsules to neutralize defenders.
- **Safe Looting:** Targets only pills classified as “safe” by the Voronoi analysis.
- **Positioning:** Moves toward formation points if no safe targets are available.

The Defender Tree follows a similar prioritized selector-based structure, focusing on home-side security and territorial control:

- **Powered Looting:** Allows the defender to transition into an offensive role and collect enemy pills when in a powered state.
- **Intruder Interception:** Reacts to suspected enemy Pacman agents within the home territory to initiate an interception.
- **Lane Blocking:** Attempts to block predicted crossing points when an enemy is estimated to be entering the agent’s assigned lane.
- **Safe Middle Pills:** Collects pills in the neutral middle zone if no immediate threats are detected.
- **Defensive Positioning:** Moves to a designated formation point or holds the drop zone to maintain a consistent defensive line.

3.2.3 Tracking and Navigation Parameters

We maintain a separate particle filter for each enemy agent. Each particle encodes a position and velocity hypothesis. For decision-making, we calculate the weighted average of all particles to generate a single estimated coordinate for the Blackboard. If the game server indicates an enemy has been captured, the filter for that agent is immediately reset to the enemy’s starting base.

For navigation, path planning is performed using the A* algorithm. To incorporate safety, we modify the A* cost function to include a penalty derived from the Voronoi danger evaluation:

$$Cost = Distance + (Danger \times PenaltyMultiplier) \quad (1)$$

In our final implementation, we utilized a *danger penalty multiplier* of 12.0. This high value ensures that agents prefer significant detours through safe territory rather than risking a direct path through an area likely controlled by an enemy ghost.

4 Experimental results

This section evaluates the performance of our multi-agent Pacman capture-the-flag system in the two official competition rounds. The main goal of the experiments was to compare how well each group performed on the evaluated maps, and to analyze whether our approach led to competitive results. Overall, our team obtained the highest combined average score across the evaluated maps, which suggests that the proposed strategy was effective in the competition setting.

4.1 Experimental setup

The evaluation was performed using two official competition rounds. Each competition used the same two maps, referred to here as Map B and Map D. In each competition, every team played one-versus-one matches against all other teams on the same map. Teams received 3 points for each win, 1 point for each draw, and 0 points for each loss.

Each match lasted 150 seconds. During a match, two teams competed to collect pills from the opponent’s side of the map. Agents acted as Pacman when entering enemy territory, where they were vulnerable to enemy ghosts, and as ghosts when staying on their own side, where they could defend against enemy Pacman agents. This scoring mechanism rewarded teams that could collect pills efficiently while still avoiding unnecessary captures and defending their own territory.

For each map, the tournament score of each group was recorded in both competitions. The final score per map was computed as the arithmetic mean of the scores obtained in Competition 1 and Competition 2. The following analysis uses the official competition results. Table 1 shows the average results for Map B across both competitions, while Table 2 shows the corresponding average results for Map D.

Rank	Group	Competition 1	Competition 2	Average
1	1	55	51	53
2	18	49	45	47
3	2	33	54	43.5
4	7	48	36	42
5	9	42	36	39
6	11	39	39	39
7	16	33	45	39
8	3	30	42	36
9	13	42	28	35
10	10	24	45	34.5
11	17	39	27	33
12	15	30	33	31.5
13	4	42	19	30.5
14	6	27	27	27
15	14	9	39	24

Table 1: Map B results across both competitions for the top 15 groups ranked by average score.

Rank	Group	Competition 1	Competition 2	Average
1	2	46	52	49
2	1	48	48	48
3	18	51	39	45
4	16	48	40	44
5	11	33	45	39
6	10	39	33	36
7	17	48	24	36
8	7	34	36	35
9	13	42	28	35
10	15	25	45	35
11	4	21	39	30
12	9	33	27	30
13	19	36	22	29
14	3	21	34	27.5
15	6	24	24	24

Table 2: Map D results across both competitions for the top 15 groups ranked by average score.

4.2 Results on Map B

On Map B, our team achieved the highest average score across the two competitions. In Competition 1, Group 1 obtained 55 points, which was the best score on the map. In Competition 2, Group 1 obtained 51 points, ranking second behind Group 2, which obtained 54 points. However, when averaging both competitions, Group 1 achieved an average score of 53 points, ranking first overall on Map B.

A likely reason for this performance is that Map B rewarded safe and consistent pill collection. Our attackers did not simply move toward the nearest pill. Instead, they used safety information from enemy estimates and Voronoi partitioning to select safer objectives and retreat options. This reduced the chance of being captured. At the same time, the defender behavior allowed the team to protect important lanes and react to enemy intruders, which helped reduce the opponent’s scoring opportunities.

4.3 Results on Map D

On Map D, our team achieved the second-highest average score. Group 2 ranked first with an average score of 49 points, while Group 1 ranked second with an average score of 48 points. Our performance was very stable across the two competitions, with 48 points in Competition 1 and 48 points again in Competition 2.

This result is important because it shows that our method generalized well to a second map. While Group 2 achieved a slightly higher average on Map D, our team had the most consistent performance among the top groups on this map. Several other groups showed larger differences between the two competitions. For example, Group 18 scored 51 points in Competition 1 but 39 points in Competition 2, while Group 17 scored 48 points in Competition 1 but 24 points in Competition 2.

The stability of our result suggests that the system was not overly tuned to a single competition run. However, the fact that Group 2 obtained the highest average on Map D suggests that there was still room for improvement, especially in increasing offensive efficiency without sacrificing safety.

4.4 Overall comparison

Considering the average scores across both maps, Group 1 achieved the highest combined score. Our average score was 53 points on Map B and 48 points on Map D, giving a combined map average of 101 points. Group 2 obtained 43.5 points on Map B and 49 points on Map D, giving a combined average of

92.5 points. Group 18 obtained 47 points on Map B and 45 points on Map D, giving a combined average of 92 points.

A common pattern among the top three groups was the use of two main roles: attackers and defenders. This suggests that explicitly dividing the team into agents responsible for stealing pills and agents responsible for protecting the home side was an effective strategy. Attackers allowed the team to maintain offensive pressure and collect points, while defenders reduced the opponent's ability to score by guarding important lanes and reacting to incoming enemies. This role separation appears to be more effective than treating all agents identically, since the game requires both scoring and protection at the same time.

The main difference between the top-performing approaches was how the agents selected their current behavior. Group 2 used decision trees to choose between different agent modes, while our group used behavior trees to organize the logic for attacking, defending, retreating, chasing, and end-game behavior. Both approaches allowed agents to switch between modes depending on the game state, but behavior trees gave our implementation a modular structure where new behaviors could be added without rewriting the entire decision logic.

One important feature of our solution was a dedicated end-game strategy. To the best of our knowledge, our group was the only one among the top teams that implemented a different behavior specifically for the final part of the match. Instead of continuing normal pill collection until the end, our agents rushed toward the two capsules and consumed them in sequence. This strategy could create a final window of about 20 seconds where our agents could collect pills more aggressively without risk of being captured. This was especially useful because the final seconds of a match are critical: pills collected near the end can change the result, while being captured late leaves little time to recover.

Another smaller but useful behavior improvement was the defender lane-blocking strategy. When an enemy position was detected or estimated, defenders mirrored the opponent's position along the map's z-axis and moved to the corresponding lane on our side. This allowed defenders to block likely incoming paths without crossing into dangerous territory. As a result, defenders could pressure enemy Pacman agents while still remaining relatively safe as ghosts on our side of the map.

These results indicate that our solution was not only strong on one specific map, but also robust across the evaluated environments. The main strength of the method appears to be the balance between attack, defense, and safety.

The results therefore support the central idea of this report: in adversarial multi-agent navigation, strong performance depends not only on shortest-

path planning, but also on behavior selection, role coordination, and timing. The best-performing behavior was not a single algorithm in isolation, but the integration of high-level decision making with safety-aware path planning, enemy-position estimation, and match-phase-specific strategies.

5 Summary and Conclusions

In this report, we presented a multi-agent navigation system for the Pacman capture-the-flag game, integrating modular behavior trees, A* path planning, and particle-filter-based state estimation. Our approach successfully addressed the challenges of adversarial coordination and partial observability by combining high-level decision-making with a Voronoi-derived safety layer.

The main contribution of this work is the effective integration of safety-aware planning with specialized roles. A key finding was the success of the defender lane-blocking strategy, which utilized axis-mirroring to intercept opponents while minimizing risk. The robustness of this system was validated by achieving the highest combined average score in competition rounds, demonstrating that performance in dynamic environments depends on the synergy between precise estimation and adaptive behavior selection.

5.1 Future Work

Future research could improve performance by incorporating learning-based opponent models to better predict adversarial intent during matches. Additionally, transitioning from static to dynamic role assignment—allowing agents to swap roles based on real-time game pressure—could further enhance scoring potential. Finally, while developed for a game environment, the modular logic and safety scoring techniques employed here are directly applicable to real-world multi-robot domains such as warehouse automation and search-and-rescue.

References

- [1] Franz Aurenhammer. Voronoi diagrams—a survey of a fundamental geometric data structure. *ACM Computing Surveys (CSUR)*, 23(3):345–405, 1991.

-
- [2] Howie M Choset, Kevin M Lynch, Seth Hutchinson, George Kantor, Wolfram Burgard, Lydia E Kavraki, and Sebastian Thrun. *Principles of robot motion: theory, algorithms, and implementations*. MIT press, 2005.
 - [3] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2017.
 - [4] Frank Dellaert, Dieter Fox, Wolfram Burgard, and Sebastian Thrun. Monte carlo localization for mobile robots. In *Proceedings of the IEEE International Conference on Robotics and Automation*, 1999.
 - [5] Jialun Hu, Hanlin Niu, Alexander S York, Hyondong Shin, Ran Etziony, and Alireza Rezaei. Voronoi-based multi-robot autonomous exploration in unknown environments. *IEEE Access*, 8:161113–161123, 2020.
 - [6] Matteo Iovino, Edvards Scukins, Jonathan Styrud, Petter Ögren, and Christian Smith. A survey of behavior trees in robotics and ai. *Robotics and Autonomous Systems*, 154:104096, 2022.
 - [7] Sebastian Thrun. Particle filters in robotics. In *Proceedings of the Eighteenth Conference on Uncertainty in Artificial Intelligence*, pages 511–518, 2002.
 - [8] Yuquan Wang, Michele Colledanchise, Alejandro Marzinotto, and Petter Ögren. A distributed convergent solution to the ambulance positioning problem on a streetmap graph. In *IFAC Proceedings Volumes (19th IFAC World Congress)*, volume 47, pages 9190–9196. Elsevier, 2014.